

The Quantum Approximate Optimization Algorithm

Construction, worked examples, and where it actually stands

Lecture 7

September 29, 2026

Quantum Approximation Optimization Algorithm (QAOA)

- QAOA is a variational quantum algorithms specifically designed to solve combinatorial optimization problems
- The objective is to identify the quantum state that minimizes the expected values of a Hamiltonian which encapsulates the cost function of the problem at hand
- QAOA uses an alternating sequence of unitary transformations, which are derived from two Hamiltonians
 - The cost Hamiltonian H_C , and
 - The mixing Hamiltonian H_B

The objective — minimize the expected cost

The problem is encoded in a cost Hamiltonian that is **diagonal** in the computational basis (L3–L4):

$$H_C |z\rangle = C(z) |z\rangle, \quad z \in \{0, 1\}^n$$

so the optimal solution is the ground state of H_C .

QAOA prepares $|\gamma, \beta\rangle$ and measures its expected cost

$$F_p(\gamma, \beta) = \langle \gamma, \beta | H_C | \gamma, \beta \rangle = \sum_z p(z) C(z),$$

The QAOA problem

$$\min_{\gamma, \beta} \langle \gamma, \beta | H_C | \gamma, \beta \rangle$$

$2p$ parameters, whatever n is.

Takeaway: QAOA is VQE with the ansatz handed to you by the problem — and with a bitstring, not a number, as the deliverable.

E. Farhi, J. Goldstone, S. Gutmann (2014). “A quantum approximate optimization algorithm.” arXiv:1411.4028

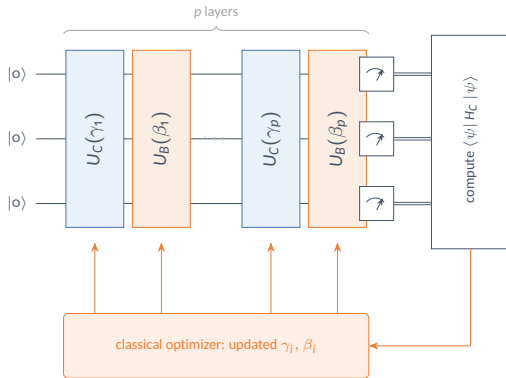
QAOA Circuit

- The circuit repeats two blocks p times:

$$U_C(\gamma) = e^{-i\gamma H_C}$$

$$U_B(\beta) = e^{-i\beta H_B}$$

- U_C stamps a phase on each bitstring; U_B moves amplitude between them
- γ_k, β_k are the variational parameters, optimized classically
- Every iteration re-runs the whole circuit and re-estimates $\langle H_C \rangle$



Cost Hamiltonian

- The cost Hamiltonian H_C encodes the objective function of the optimization problem
- Assigns the “energy” (or “cost”) to a specific configuration in the system
- As such, the goal is to find the configuration that minimizes this energy (i.e., cost)

Mixing Hamiltonian

- The mixing Hamiltonian H_B facilitates transitions between different cur configurations
- This will aid the system in exploring the space of potential solutions
- A common choice for H_B is

$$H_B = \sum_i X_i$$

- Where X_i is the Pauli-X operator on the i-th qubit

The QAOA state, written out

The state and the objective

$$|\gamma, \beta\rangle = \underbrace{e^{-i\beta_p H_B} e^{-i\gamma_p H_C}}_{\text{layer } p} \cdots \underbrace{e^{-i\beta_1 H_B} e^{-i\gamma_1 H_C}}_{\text{layer } 1} |+\rangle^{\otimes n}, \quad F_p(\gamma, \beta) = \langle \gamma, \beta | H_C | \gamma, \beta \rangle.$$

Reading the formula.

- Operators compose right to left, but a *circuit* draws layer 1 on the left. The two orderings are mirror images — the usual source of sign errors.
- H_C is diagonal, so $e^{-i\gamma H_C}$ merely stamps a phase $e^{-i\gamma C(z)}$ on each $|z\rangle$. It moves no probability.
- $H_B = \sum_i X_i$ is not diagonal. It is the *only* thing that moves amplitude between bitstrings — which is why it must not commute with H_C .

$2p$ numbers, whatever n is.

- The search space is $\gamma, \beta \in \mathbb{R}^p$ — independent of the problem size. The circuit grows with n ; the optimization problem does not.
- If $C(z)$ is integer-valued (Max-Cut is), F_p has period 2π in each γ_k ; $e^{-i\beta H_B}$ has period π in each β_k .
- H_C , H_B and $|+\rangle^{\otimes n}$ are all real, so $F_p(-\gamma, -\beta) = F_p(\gamma, \beta)$: half the domain is redundant.

Takeaway: Fixing the convention now costs one slide and saves an afternoon. Everything below uses *cost first, mixer second*, with $|+\rangle^{\otimes n}$ as the ground state of $-H_B$.

Max-Cut end to end

Take the 5-vertex graph from L3, all weights 1.

1. **Cost Hamiltonian.** Each edge contributes

$$H_C = \frac{1}{2} \sum_{(i,j) \in E} (\mathbb{I} - Z_i Z_j),$$

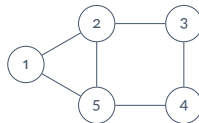
which counts cut edges: $Z_i Z_j = -1$ exactly when i and j are on opposite sides.

2. **Mixer.** $H_B = \sum_i X_i$, whose ground state $|+\rangle^{\otimes n}$ is the uniform superposition over all 2^n cuts.

3. **Circuit.** $|\gamma, \beta\rangle = \prod_{k=1}^p e^{-i\beta_k H_B} e^{-i\gamma_k H_C} |+\rangle^{\otimes n}$.

4. **Measure** in the computational basis, average $\langle H_C \rangle$, hand it to the optimizer.

Takeaway: The circuit depth per layer is set by the *graph*, not by the problem size alone — which is why hardware connectivity decides what you can actually run.



Each edge (i, j) becomes one ZZ rotation: CNOT, $R_Z(2\gamma)$, CNOT.

Six edges $\Rightarrow$ twelve CNOTs per layer.

Forming H_C , term by term

1. **Encode the partition.** Give vertex i a spin $s_i = +1$ on one side, $s_i = -1$ on the other ($s_i = 1 - 2z_i$ for a bit z_i).

2. **Count one edge.** Edge (i, j) is cut exactly when $s_i s_j = -1$, so its indicator is $\frac{1}{2}(1 - s_i s_j)$.

3. **Sum over edges.**

$$C(s) = \sum_{(i,j) \in E} \frac{1}{2}(1 - s_i s_j) = \frac{|E|}{2} - \frac{1}{2} \sum_{(i,j) \in E} s_i s_j$$

4. **Promote spins to operators, $s_i \rightarrow Z_i$:**

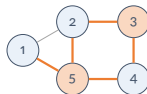
$$H_C = \frac{1}{2} \sum_{(i,j) \in E} (\mathbb{I} - Z_i Z_j) = \frac{|E|}{2} \mathbb{I} - \frac{1}{2} \sum_{(i,j) \in E} Z_i Z_j$$

H_C is **diagonal**: $H_C |z\rangle = C(z) |z\rangle$, so its eigenvalues *are* the cut values.

This instance ($|E| = 6$):

$$H_C = 3\mathbb{I} - \frac{1}{2}(Z_1 Z_2 + Z_2 Z_3 + Z_1 Z_5 + Z_4 Z_5 + Z_2 Z_5 + Z_3 Z_4)$$

Takeaway: Building H_C is bookkeeping, not physics: one $Z_i Z_j$ per edge, and a constant that shifts every energy equally (L3).



$z = 00101$: side $\{3, 5\}$ cuts 5 of 6 edges

Check that state. $s = (+, +, -, +, -)$:

$$\sum s_i s_j = 1 - 1 - 1 - 1 - 1 - 1 = -4$$

$$H_C |z\rangle = \left(3 - \frac{1}{2}(-4)\right) |z\rangle = 5 |z\rangle \quad \checkmark$$

The whole spectrum (32 states):

cut value	0	2	3	4	5
states	2	8	12	6	4

Mean = $3 = |E|/2$: the average over all cuts is exactly the constant term.

Forming H_B , and what one layer costs

The mixer. $H_B = \sum_i X_i$, a sum of single-qubit terms, so its exponential factorises into one rotation per qubit:

$$e^{-i\beta H_B} = \bigotimes_{i=1}^n e^{-i\beta X_i} = \bigotimes_{i=1}^n R_X(2\beta)$$

The starting state. $|+\rangle^{\otimes n}$ is the ground state of $-H_B$ and the uniform superposition over all 2^n cuts. It is prepared with one H per qubit, and in it

$$\langle + |^{\otimes n} H_C | + \rangle^{\otimes n} = \frac{|E|}{2} = 3,$$

the random-guessing value — QAOA starts exactly there and pushes upwards.

Why H_B must not commute with H_C . H_C is diagonal, so on its own it only stamps phases: the probabilities never move. X anticommutes with Z , so $[H_C, H_B] \neq 0$ and the mixer turns those phases into amplitude.

One layer on this instance

Block	Gates	Count
state prep	H	5
$e^{-i\gamma H_C}$	CNOT, $R_Z(2\gamma)$, CNOT	12 + 6
$e^{-i\beta H_B}$	$R_X(2\beta)$	5
per layer	two-qubit gates	12
$p = 3$	two-qubit gates	36

Parameters: $2p$, whatever n is — two numbers at $p = 1$, six at $p = 3$.

Depth is set by the graph: one ZZ rotation per *edge*, so a denser graph costs more per layer even at the same n (next frame).

Compiling the phase separator

Every term of H_C commutes with every other, so $e^{-i\gamma H_C}$ factorises into independent two-qubit rotations that the compiler may reorder freely:

$$e^{-i\theta Z_i Z_j / 2} = \text{CNOT}_{ij} (\mathbb{I} \otimes R_z(\theta)) \text{CNOT}_{ij}, \quad R_z(\theta) = e^{-i\theta Z / 2}.$$



Two CNOTs and one z-rotation, per edge, per layer.

Then reality: the qubits must be adjacent.

- A device offers a fixed *coupling graph*. A problem edge that is not a coupling edge has to be routed there, and a SWAP costs three CNOTs.
- Compiling is therefore a graph-embedding problem, exactly as in L5 — the annealer's minor embedding and the gate model's routing are the same difficulty wearing different clothes.

Coupling graph	ZZ available on	Cost of a dense problem graph
all-to-all (small ion traps)	any pair	none: $ E $ rotations per layer
grid / heavy-hex (superconducting)	nearest neighbours only	a swap network, $\mathcal{O}(n)$ depth per layer
line	nearest neighbours only	$\mathcal{O}(n^2)$ SWAPs to touch every pair

The catch: two-qubit error accumulates with gate count, so the usable depth is roughly $p_{\max} \approx 1/(\epsilon_2 \times \text{two-qubit gates per layer})$. Routing overhead does not just slow the circuit down — it lowers the p you are allowed to ask for.

Constrained problems: penalties, or a different mixer

The L4 route: penalties. A constraint $\sum_i x_i = k$ becomes $\lambda(\sum_i x_i - k)^2$ in the QUBO.

- Expanding the square produces a term for every pair (i, j) : the penalty is a complete graph on the constrained variables, so $|E|$ jumps to $\mathcal{O}(n^2)$ and the phase separator becomes the dominant cost.
- λ must exceed the largest objective difference or infeasible strings win — but a large λ squeezes the objective differences into a small band of a wide spectrum, and those are the differences the optimizer has to resolve.
- Most of the 2^n strings remain infeasible, and the mixer keeps pushing amplitude into them. You pay shots to sample answers you will discard.

The other route: constrain the mixer. Replace $\sum_i X_i$ with an XY mixer on a ring or a complete graph,

$$H_{XY} = \frac{1}{2} \sum_{(i,j)} (X_i X_j + Y_i Y_j),$$

which commutes with the Hamming weight $\sum_i \frac{1}{2}(\mathbb{I} - Z_i)$.

- Start in a Dicke state $|D_k^n\rangle$ — the uniform superposition over weight- k strings — and the evolution never leaves the feasible subspace.
- H_C then encodes the objective only. No λ to tune, no wasted samples.
- The price: $|D_k^n\rangle$ needs a non-trivial preparation circuit, the XY terms do not commute with each other so their exponential must itself be Trotterized, and the mixer is now two-qubit gates rather than one.

This is the step from “quantum approximate optimization algorithm” to *quantum alternating operator ansatz*: the mixer is any operator that preserves the feasible set.

The catch: the mixer must also connect the whole feasible subspace. A conserving mixer that is reducible leaves you trapped in whichever sector the initial state happened to occupy.

Case study: the Vehicle Routing Problem

The constrained problem of L2, taken through exactly the machinery of this lecture.

Instance. n locations on a complete directed graph, node o the depot, weights w_{ij} , and k vehicles.

Decision variables. One binary per *directed edge*,

$$x_{ij} \in \{0, 1\}, \quad i \neq j \quad \implies \quad n(n-1) \text{ variables}$$

Objective. $\min \sum_{i \rightarrow j} w_{ij} x_{ij}$

Constraints.

- ① each non-depot node has exactly one outgoing edge:
 $\sum_j x_{ij} = 1$
- ② each non-depot node has exactly one incoming edge:
 $\sum_j x_{ji} = 1$
- ③ exactly k edges leave the depot: $\sum_j x_{oj} = k$
- ④ exactly k edges return to it: $\sum_j x_{jo} = k$

Note what is *not* here: no vehicle index and no capacity. This is the vanilla VRP of L2; the capacitated version needs load variables or slack bits on top (L4).

U. Azad, B. K. Behera, E. A. Ahmed, P. K. Panigrahi, A. Farouk (2023). "Solving Vehicle Routing Problem Using Quantum Approximate Optimization Algorithm." *IEEE Trans. Intelligent Transportation Systems* 24(7), 7564–7573. doi:10.1109/TITS.2022.3172241 (preprint arXiv:2002.01351)

One qubit per variable

n	variables $n(n-1)$	qubits
4	12	12
5	20	20
6	30	30
8	56	56
10	90	90

The count does not depend on k : the number of vehicles enters only through constraints 3 and 4.

Our PA1 instance has $n = 7$, so this model needs **42** qubits — and the per-vehicle model of L2, with x_{ij}^k , needs **84**.

The VRP Hamiltonian, term by term

Each constraint becomes a squared deviation, weighted by one penalty constant $A > 0$ (L4):

$$H_{\text{VRP}} = H_A + H_B + H_C + H_D + H_E$$

$$H_A = \sum_{i \rightarrow j} w_{ij} x_{ij}$$

the objective

$$H_B = A \sum_{i=1}^{n-1} \left(1 - \sum_j x_{ij}\right)^2 \quad H_C = A \sum_{i=1}^{n-1} \left(1 - \sum_j x_{ji}\right)^2$$

leave once, enter once

$$H_D = A \left(k - \sum_j x_{0j}\right)^2 \quad H_E = A \left(k - \sum_j x_{j0}\right)^2$$

k out of, k into the depot

Careful with the names: in this paper $H_B, \dots, H_E$ are penalty terms, *not* the mixer. The mixer is still $\sum_i X_i$.

Why the squares matter. Expanding $\left(1 - \sum_j x_{ij}\right)^2$ gives a term $x_{ij}x_{ij'}$ for every pair of edges at node i : the penalties, not the objective, make the model quadratic and dense.

To spins. $x_{ij} = \frac{s_{ij}+1}{2}$ turns H_{VRP} into an Ising Hamiltonian $\sum J s s + \sum h s + \text{const}$ — what the phase separator is built from.

Choosing A. Large enough that no infeasible assignment undercuts a feasible one — of the order of the largest route cost — but no larger, or the objective differences shrink into a band the optimizer cannot resolve (L4).

The ground state is a bitstring over edges. Feasibility is enforced by energy, not by construction, so every sample is still checked classically.

VRP with QAOA: what the paper reports

(n, k)	qubits	depth p	optimizers
$(4, 2)$	12	$p \geq 12$	COBYLA
$(5, 2)$	20	6 to 40	COBYLA, Nelder–Mead, L-BFGS-B
$(5, 3)$	20	$p \geq 24$	COBYLA

All three are simulations (Qiskit), and all three recover the known optimal route once p is large enough. The reported conclusion is that performance depends not only on the optimizer, on p , or on how the parameters are initialized, **but on the instance itself**.

Read the depths again. $p \geq 24$ on 20 qubits means, per layer, one ZZ rotation for every pair of edges the penalties couple — hundreds of two-qubit gates, multiplied by 24. That is far past the noise ceiling of the previous frames.

Two ways out, both used in this course.

- **Reduce the model.** In PA1 you replace the routing model by a 6-qubit *partition* problem, solve that with QAOA, and route each cluster classically
- **Constrain the mixer** (previous frame) so the degree constraints hold by construction and A disappears

The catch: the VRP is the honest picture of applying QAOA to a real constrained problem: the variables are the easy part, the penalties set the qubit *and* depth budget, and the instance decides whether any of it works.

U. Azad, B. K. Behera, E. A. Ahmed, P. K. Panigrahi, A. Farouk (2023). *IEEE Trans. Intelligent Transportation Systems* 24(7), 7564–7573.
doi:10.1109/TITS.2022.3172241

References

Construction

- E. Farhi, J. Goldstone, S. Gutmann (2014). "A Quantum Approximate Optimization Algorithm." arXiv:1411.4028 (*preprint, never journal-published*)
- S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, R. Biswas (2019). "From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz." *Algorithms* 12(2), 34. arXiv:1709.03489
- Z. Wang, S. Hadfield, Z. Jiang, E. G. Rieffel (2018). "Quantum approximate optimization algorithm for MaxCut: a fermionic view." *Phys. Rev. A* 97, 022304. arXiv:1706.02998

Applications

- U. Azad, B. K. Behera, E. A. Ahmed, P. K. Panigrahi, A. Farouk (2023). "Solving Vehicle Routing Problem Using Quantum Approximate Optimization Algorithm." *IEEE Transactions on Intelligent Transportation Systems* 24(7), 7564–7573. doi:10.1109/TITS.2022.3172241. arXiv:2002.01351